Mole Table Wiring

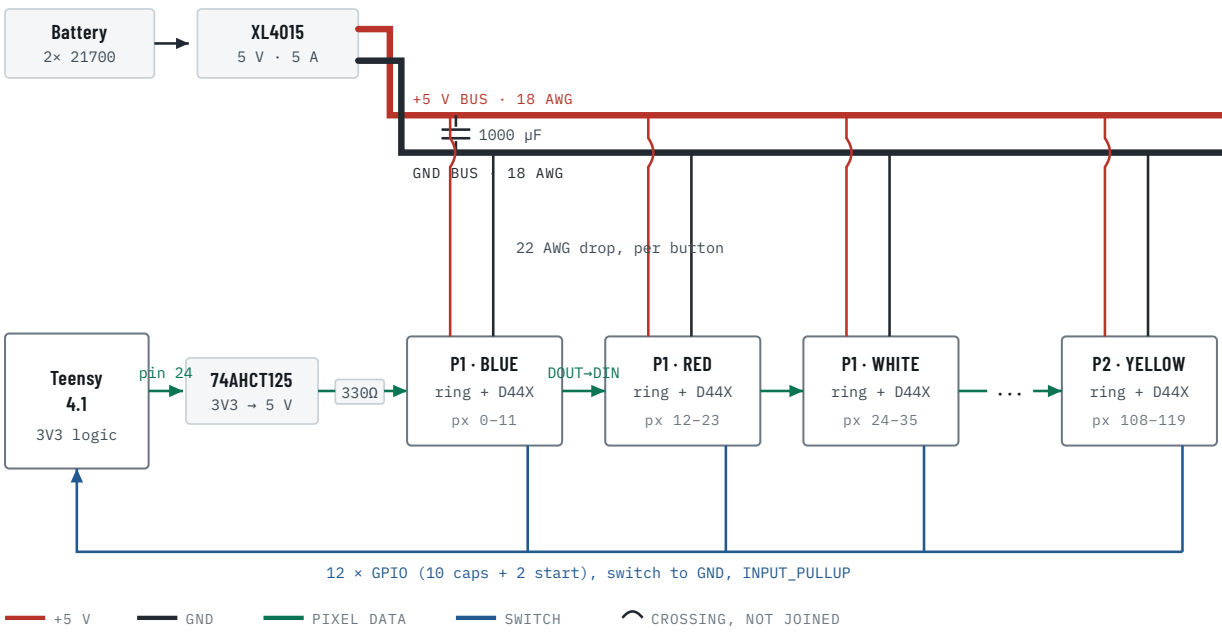
Ten 60 mm dome buttons carrying a 12-pixel NeoPixel ring and a Cherry D44X switch, plus two unlit start buttons, back to one Teensy 4.1. Pin assignments are mirrored from `firmware/WhackAMole/Config.h`.

The one rule

Power does not daisy-chain through the buttons. Only the data line does. A 12-pixel ring pulls **720 mA** at full white, so chaining 5 V through the button connectors would put **7.2 A** through the first one. Run a separate 5 V/GND bus and tap each button off it.

System topology

10 lit caps · 120 pixels

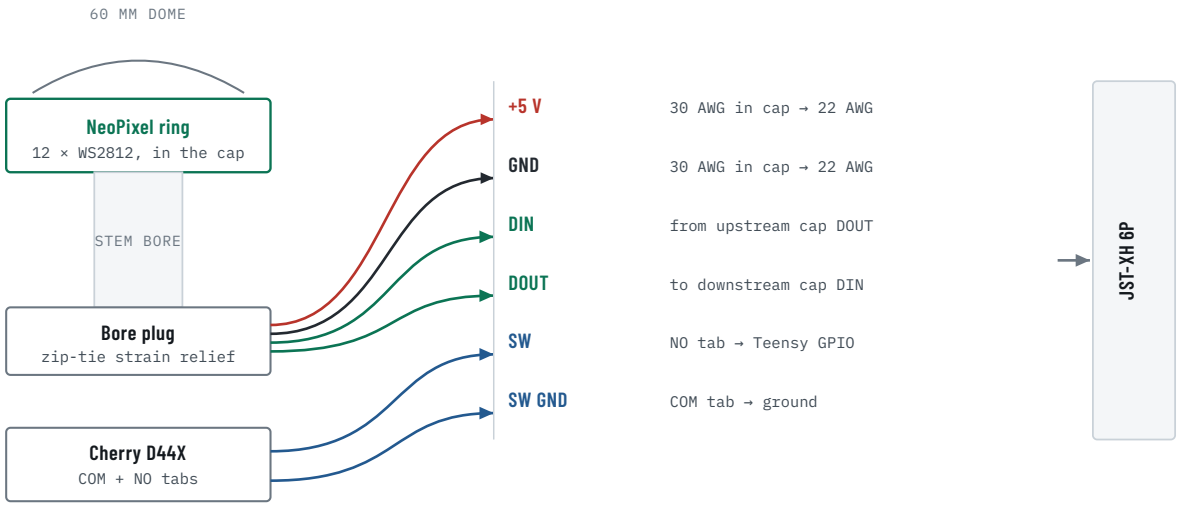


Two independent networks. Power fans out from a shared bus so no connector carries more than one ring's current; data runs series through every ring in turn, which is why one unplugged cap goes dark for everything downstream of it. The two start buttons sit outside both networks — switch only.

18 AWG bus, 22 AWG drops · 26 AWG point to point · 22 AWG returns

Inside one cap

6 conductors out



Keep the two grounds separate all the way back. A ring at full white returns 720 mA; sharing that return with the switch puts an IR offset across the input the Teensy is trying to read.

Current budget

worst case is a debug screen

CONDITION	PIXELS LIT	CURRENT	NOTES
Gameplay, two moles up	24	≈ 0.30 A	Amber, not white. The normal case.
Attract mode	0–24	≈ 0.15 A	Quiescent draw is ~1 mA per pixel.
Debug `LED ALL`, white	120	2.26 A	At `kPixelBrightness = 80`.
Same, brightness 255	120	7.20 A	Why the ceiling exists.
32×32 matrix panel	—	≈ 2.0 A	Shares the same buck.
Worst realistic total	**120**	**≈ 4.4 A**	**Against a 5 A converter.**

Measure before you trust this

The worst case leaves roughly 12% headroom on the buck, which is workable but not generous. The `LED ALL` debug screen exists precisely so you can put a meter on the 5 V rail and find out. If it runs hot or sags, drop `kPixelBrightness` — gameplay never approaches this, so lowering it costs nothing you will see.

Chain order `must match Config.h`

The physical daisy-chain order and the pixel indices in `Config.h` are the same fact written twice. Wire them in this sequence or the wrong cap lights.

POSITION	BUTTON	PIXELS	SWITCH PIN
1–5	P1 mole caps — blue, red, white, green, yellow	0–59	2, 3, 4, 5, 9
6–10	P2 mole caps — blue, red, white, green, yellow	60–119	11, 12, 14, 15, 16
—	P1 start · P2 start (no lamp)	none	22 · 25

The start buttons are switch-only

They are plain arcade buttons with no lamp, so they contribute a switch and nothing else: two conductors, no ring, and no place in the pixel chain. `kPixelCount` is `kButtonsPerPlayer × 2 × kPixelsPerButton` — 120, not 144. Allocating pixels for them would shift nothing electrically but would leave the chain expecting two rings that are not there.

Teensy 4.1 pins from Config.h

FUNCTION	PINS	NOTES
Display link, Serial1	`0 RX · 1 TX`	To the Matrix Portal at 115200.
Pixel data	`24`	Serial6 TX. WS2812Serial only drives certain pins.
P1 buttons	`2, 3, 4, 5, 9`	All `INPUT_PULLUP`, switch to ground.
P2 buttons	`11, 12, 14, 15, 16`	
Start buttons	`22 · 25`	
Reserved for audio	`6, 7, 10, 18, 19, 20, 21, 23`	Do not use. Pin 7 is I2S data, which is why the display moved off Serial2.
PRNG seed	`17`	Left floating deliberately.

Parts beyond the buttons

PART	QTY	WHY
74AHCT125 quad level shifter	1	Teensy is 3.3 V; WS2812B wants 0.7×VDD on data. The usual cause of flaky pixel builds. Tie unused inputs and `OE` low.
330–470 Ω resistor	1	Series on the data line at the first pixel.
1000 µF electrolytic, ≥ 6.3 V	1	Across 5 V and ground at the bus entry.
JST-XH 6-pin pairs	10	One per lit cap, so it unplugs for service.
JST-XH 2-pin pairs	2	The start buttons — switch only.
Terminal strip or lever nuts	1	The 5 V/GND bus taps. Ten pairs.
30 AWG silicone wire	—	Inside the lit caps only; it has to survive the 3 mm of plunger travel.
0.187" female spade crimps	24	Onto the D44X tabs.

Build order `test as you go`

01 **Wire one cap completely**
Ring, switch, connector, all six conductors. Getting the first one right tells you the gauge and connector choices work before you repeat them nine times.

02 **Run it on the bench**
Level shifter, resistor, cap, one button. Flash the firmware with `WAM_LIGHTS_PIXELS 1` and open the debug `LED WALK` screen — one ring is enough to prove the data path.

03 **Build the power bus first**
Before any daisy-chaining. Confirm 5 V at every tap with nothing else connected.

04 **Add caps to the chain one at a time**
Each addition is a new failure point, and `LED WALK` names the cap it is lighting. A break shows up immediately as everything downstream going dark.

05 **Meter the rail on `LED ALL`**
The only screen that reaches worst-case current. Do this before the table sees players, not after.

06 **Verify every switch on `BUTTONS`**
Press all twelve, the two start buttons included, and confirm each decodes as the right name. This is what catches a swapped pair in the loom.